

TRABALHO FINAL

Programação Orientada a Objetos — C#

Gestão de Contas Bancárias

Aplicação de Consola em C#

António Sousa Nº 35699

2025 / 2026

Índice

1. Introdução	3
2. Estrutura do Projeto	3
2.1 Movimento.cs	3
2.2 Conta.cs	3
2.3 Cliente.cs	4
2.4 Program.cs	4
3. Funcionalidades	5
4. Tratamento de Erros	5
5. Conceitos de POO Aplicados	5
6. Conclusão	6

1. Introdução

Este relatório descreve o desenvolvimento de uma aplicação de consola em C# para a gestão simples de contas bancárias, realizada no âmbito da unidade curricular de Programação Orientada a Objetos.

O objetivo principal foi aplicar os conceitos fundamentais de POO — classes, objetos, encapsulamento e coleções — num contexto próximo da realidade.

2. Estrutura do Projeto

O projeto é composto por 4 ficheiros C# independentes, cada um com a sua responsabilidade:

Ficheiro	Responsabilidade
Movimento.cs	Guarda os dados de cada transação: data, tipo e valor
Conta.cs	Gere o saldo, depósitos, levantamentos e extrato
Cliente.cs	Associa o nome e NIF a uma lista de contas
Program.cs	Menu interativo e toda a lógica de navegação

2.1 Movimento.cs

Representa uma operação realizada na conta. Cada objeto guarda a data e hora automática, o tipo (Depósito, Levantamento, Abertura) e o valor.

```
class Movimento
{
    public DateTime Data { get; private set; }
    public string Tipo { get; private set; }
    public decimal Valor { get; private set; }

    public Movimento(string tipo, decimal valor)
    {
        Data = DateTime.Now;
        Tipo = tipo;
        Valor = valor;
    }
}
```

2.2 Conta.cs

Representa uma conta bancária com número único gerado automaticamente (ex: PT1001). Contém o saldo atual e o histórico de movimentos.

Métodos disponíveis:

- Depositar(decimal valor) — adiciona o valor ao saldo e regista o movimento
- Levantar(decimal valor) — subtrai o valor com validação de saldo suficiente
- MostrarExtrato() — apresenta todos os movimentos em tabela formatada

```
class Conta
```

```
{
    private static int _contador = 1000;
    public string Numero { get; private set; }
    public decimal Saldo { get; private set; }
    private List<Movimento> _movimentos = new List<Movimento>();

    public Conta(decimal saldoInicial = 0) { ... }
    public void Depositar(decimal valor) { ... }
    public void Levantar(decimal valor) { ... }
    public void MostrarExtrato() { ... }
}
```

2.3 Cliente.cs

Representa um cliente do banco identificado por nome e NIF. Cada cliente pode ter várias contas associadas através do método AbrirConta().

```
class Cliente
{
    public string Nome { get; private set; }
    public string NIF { get; private set; }
    public List<Conta> Contas { get; private set; }

    public Cliente(string nome, string nif) { ... }
    public Conta AbrirConta(decimal saldoInicial = 0) { ... }
}
```

2.4 Program.cs

Contém o menu principal e os métodos auxiliares de navegação (EscolherCliente, EscolherConta). É o ponto de entrada da aplicação e coordena a interação com o utilizador.

3. Funcionalidades

A aplicação apresenta um menu em consola com as seguintes opções:

Opção	Descrição
1 — Criar cliente	Regista nome e NIF de um novo cliente
2 — Abrir conta	Cria conta associada a um cliente com saldo inicial
3 — Depositar	Adiciona valor ao saldo da conta selecionada
4 — Levantar	Retira valor com verificação de fundos
5 — Ver extrato	Mostra histórico completo de movimentos
0 — Sair	Termina a aplicação

O menu limpa o ecrã entre cada operação (`Console.Clear()`) e aguarda que o utilizador pressione ENTER antes de voltar ao menu principal, mantendo a interface sempre organizada.

4. Tratamento de Erros

A aplicação valida todas as entradas do utilizador antes de executar qualquer operação:

- Saldo insuficiente — o levantamento é cancelado e é apresentada mensagem de erro
- Valor negativo ou zero — a operação é rejeitada imediatamente
- Texto em campo numérico — tratado com `TryParse`, evitando crash da aplicação
- Campos em branco — validados antes de criar cliente ou conta
- Sem clientes ou sem contas — informa o utilizador e regressa ao menu

5. Conceitos de POO Aplicados

- Classes e Objetos — cada entidade (Movimento, Conta, Cliente) tem o seu próprio ficheiro .cs
- Encapsulamento — atributos com `get` privado; saldo apenas alterado pelos métodos da classe
- Responsabilidade única — cada classe trata apenas da sua própria lógica
- Coleções — `List<T>` para guardar múltiplos clientes, contas e movimentos
- Separação de ficheiros — facilita a manutenção e leitura do código

6. Conclusão

A aplicação foi desenvolvida com sucesso, cumprindo todos os requisitos do enunciado. A separação do código em múltiplos ficheiros — um por classe — demonstra uma aplicação mais madura dos princípios de Programação Orientada a Objetos, facilitando a leitura, manutenção e possível extensão futura do projeto.

Foram implementadas todas as funcionalidades essenciais de gestão bancária com tratamento adequado de erros e uma interface de consola limpa e organizada.